

MAGÍSTER EN CIENCIA DE DATOS E INTELIGENCIA ARTIFICIAL

MCDI500 · Programación para la Ciencia de Datos

Fase 3 · Semana 2

Núcleo algorítmico, eficiencia y programación orientada a objetos

Del pipeline que funciona al sistema que se puede mantener

Lo que veremos hoy

Cuatro bloques, del algoritmo a la arquitectura

1

Programación estructurada y recursiva

Descomponer el problema. Cuándo la recursión ayuda y cuándo estorba.

2

Complejidad y eficiencia

Medir tiempo y memoria. Comparar implementaciones con evidencia.

3

Programación orientada a objetos

Encapsulamiento, herencia y polimorfismo, aplicados al pipeline.

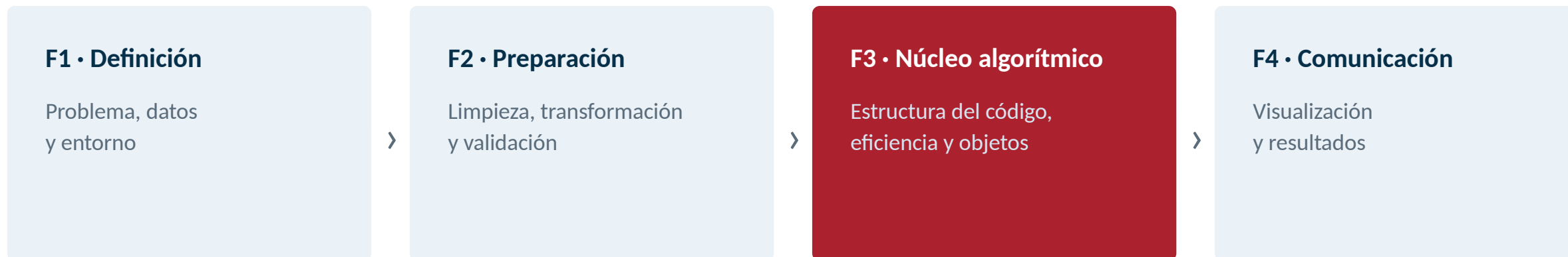
4

Patrones de diseño

Strategy, Factory, Observer y Singleton: nombres para lo que ya hicieron.

Dónde estamos en el proyecto

Las cuatro fases y qué aporta cada una



El cambio de foco

Hasta la Fase 2 el objeto de estudio eran los datos. Desde la Fase 3 el objeto de estudio es el código.

No se cambia de conjunto ni se rehace el pipeline: se reorganiza lo que ya existe para que se pueda probar, reemplazar y reutilizar.

Qué se evalúa esta semana

Tres hitos encadenados, y el segundo alimenta al tercero

Foro técnico

Discusión con una medición propia sobre la mesa. Una intervención inicial, dos de seguimiento y una síntesis.

0%

Formativa 3

Notebook con scripts y mediciones de complejidad. Retroalimentación sin nota, previa a la sumativa.

0% • 69 pts

Sumativa 2

Núcleo algorítmico con clases, herencia, polimorfismo y patrones de diseño.

20% • 69 pts

El dato que conviene tener presente

En la Sumativa 2, de los 69 puntos: 45 están en el notebook y el código, 18 en el informe y 6 en el repositorio. Más de una cuarta parte de la nota no depende de programar, sino de explicar lo que programaron.

01

Programación estructurada y recursiva

Descomponer el problema antes de escribir la primera línea

De celdas sueltas a funciones

El primer paso, y el que más rinde

```
# Antes: el orden está implícito
df = pd.read_csv(ruta)
df = df.drop(columns=['id'])
m = df['bmi'].median()
df['bmi'] = df['bmi'].fillna(m)
df = pd.get_dummies(df, ['work_type'])
```

```
# Después: cada paso tiene nombre
def cargar(ruta): ...
def quitar_id(df): ...
def imputar(df, columna): ...
def codificar(df, columnas): ...

df = codificar(imputar(quitar_id(cargar(r))))
```

- Cada función hace una sola cosa y declara qué recibe y qué devuelve
- Se puede probar por separado: si algo falla, se sabe dónde
- Se reutiliza en la Fase 4 sin copiar y pegar
- El orden deja de ser implícito y pasa a estar escrito

Recursión: una función que se llama a sí misma

Siempre tiene dos partes, y las dos son obligatorias

Caso base

Dice cuándo parar. Sin él, la función se llama a sí misma para siempre hasta que Python la detiene con un error.

Caso recursivo

Reduce el problema a una versión más pequeña del mismo problema, y vuelve a llamarse con ella.

```
def aplanar(estructura, prefijo=''):
    plano = {}
    for clave, valor in estructura.items():
        compuesta = f'{prefijo}.{clave}' if prefijo else clave
        if isinstance(valor, dict):
            plano.update(aplanar(valor, compuesta)) # caso recursivo
        else:
            plano[compuesta] = valor                # caso base
```

¿Su proyecto necesita recursión?

La respuesta honesta vale lo mismo que la implementación

Sí, cuando...

- La profundidad no se conoce al escribir el código
- Hay que recorrer una jerarquía de niveles variables
- El problema se divide en subproblemas del mismo tipo
- Se explora un árbol de decisiones

No, cuando...

- La profundidad es fija y conocida
- Un bucle resuelve lo mismo con menos líneas
- Se agrega solo para cumplir el requisito
- El volumen puede agotar la pila de llamadas

Si su proyecto no tiene un problema recursivo natural

Declárenlo en el informe y expliquen por qué el enfoque iterativo es preferible. La rúbrica acepta recursividad o división funcional: una decisión argumentada vale lo mismo que implementarla.

Recursión no es sinónimo de eficiencia

La misma idea, escrita de dos formas, con costos muy distintos

```
def fib_ingenua(n):  
    if n < 2:  
        return n  
    return (fib_ingenua(n-1)  
            + fib_ingenua(n-2))
```

```
def fib_memo(n, cache=None):  
    cache = cache or {}  
    if n in cache: return cache[n]  
    r = n if n < 2 else (fib_memo(n-1, cache)  
                        + fib_memo(n-2, cache))  
    cache[n] = r; return r
```

22,6×

más rápida con n = 18

285,7×

más rápida con n = 22

1.445,4×

más rápida con n = 26

El cambio entre una versión y otra son dos líneas. Lo que la hace eficiente no es la recursión: es no repetir trabajo.

02

Complejidad y eficiencia

La eficiencia no se declara: se mide

Complejidad: cómo crece el costo

Lo que importa no es cuánto tarda, sino cómo cambia al crecer los datos

Buscar recorriendo

Se revisa fila por fila hasta encontrar. Con 100.000 filas y el dato al final, hay que mirar 100.000.

Buscar en un diccionario

Construirlo cuesta una pasada. Cada búsqueda posterior es prácticamente instantánea, sin importar el tamaño.

El mismo cálculo, medido sobre tamaños crecientes:

0,4×

1.000 filas

1,1×

5.000

2,0×

20.000

2,5×

50.000

En 1.000 filas la versión vectorizada fue más lenta. La ventaja aparece cuando el volumen crece.

Las tres herramientas para medir

Cada una sirve para algo distinto

```
time.perf_counter()
```

Tiempo de un intervalo

Etapas del pipeline, bloques largos

```
timeit.timeit()
```

Tiempo promedio repitiendo

Fragmentos pequeños y rápidos

```
tracemalloc
```

Memoria que asigna Python

Cuando el conjunto es grande

No usen `time.time()`: tiene menos resolución y se ve afectado por ajustes del reloj del sistema.

Tres reglas para que la medición valga

Sin estas tres, el número que reporten no prueba nada

1

Repitan y conserven el MÍNIMO

No el promedio. Los tiempos altos suelen deberse a que el computador estaba haciendo otra cosa, no a su código.

2

Midan sobre el tamaño real

En mil filas todo es instantáneo y la comparación no distingue nada. Usen el volumen de su conjunto.

3

Comprueben que dan el mismo resultado

Una versión más rápida que entrega otra respuesta no es una optimización: es un error.

```
assert version_a(df) == version_b(df), 'Las versiones no coinciden'
```

03

Programación orientada a objetos

Tres principios, aplicados al pipeline que ya tienen

Antes de hablar de clases: el problema real

Esto le pasó a varios equipos en la Fase 2

```
mediana_bmi = entrenamiento['bmi'].median()  
media_age   = entrenamiento['age'].mean()  
desv_age    = entrenamiento['age'].std()  
categorias_work = sorted(entrenamiento['work_type'].unique())  
# ... y ocho variables más flotando en el cuaderno
```

Las preguntas incómodas

¿Se calculó con el entrenamiento o con el conjunto completo?
¿Alguien la sobrescribió en la celda 30?
¿Y si mañana hay dos escaladores?

Lo que una clase resuelve

Guarda el dato junto al código que lo usa, en un lugar con nombre.

Nadie lo puede pisar por accidente, y si hay dos escaladores cada uno tiene el suyo.

Clase y objeto: el molde y sus copias

Una clase no es un dato: describe qué guarda algo y qué sabe hacer

La clase es el formulario en blanco

Define qué campos existen. No tiene valores propios: describe la forma que tendrán las copias.

Escribir `class Contador` no crea ningún contador.

El objeto es un formulario llenado

Tiene sus propios valores en esos campos. Dos objetos de la misma clase son independientes.

`Contador()` sí crea uno, con su propia memoria.

La analogía completa

El formulario en blanco dice que hay un campo «nombre» y un campo «edad». No dice Ana ni 34.

Si corrigen el formulario en blanco y agregan un campo, todos los que se llenen desde ahí lo tendrán. Pero corregir un formulario ya llenado no afecta a los demás.

Anatomía de una clase, parte por parte

Cinco elementos, y ninguno es difícil por separado

```
class Contador:

    def __init__(self):
        self.total = 0

    def sumar(self, cantidad=1):
        self.total = self.total + cantidad
        return self.total
```

`class Contador:`

La clase. El nombre va en mayúscula inicial, por convención.

`__init__`

El constructor. Python lo ejecuta solo al crear el objeto.

`self`

El objeto que se está creando o usando.

`self.total`

Un atributo: un dato que el objeto guarda.

`def sumar(...)`

Un método: una operación que el objeto sabe hacer.

Los dos guiones bajos de `__init__`

No son decorativos. Python usa ese nombre exacto para saber cuál método ejecutar al crear un objeto. Si lo escriben `_init_` con un guion, no se ejecuta y nadie avisa.

Qué pasa cuando escriben `c = Contador()`

Cuatro cosas, en este orden

1

Python reserva memoria

Crea un objeto vacío del tipo `Contador`. Todavía no tiene ningún dato.

2

Llama a `__init__` automáticamente

Ustedes no lo escriben. Python lo hace solo, y le pasa el objeto recién creado.

3

`__init__` deja listos los atributos

`self.total = 0` escribe un dato dentro de ese objeto, no fuera.

4

El objeto queda asignado a `c`

Desde ahí, `c.total` lee el dato y `c.sumar()` ejecuta el método sobre ese objeto.

```
c = Contador()      # los paréntesis son los que disparan todo esto
```

Dos objetos, dos memorias separadas

El punto que más cuesta al empezar

```
c1 = Contador()    # primer formulario llenado
c2 = Contador()    # segundo, independiente del primero

c1.sumar(10)       # solo cambia c1
```

c1

total = 10

c2

total = 0

El molde es el mismo. Los datos, no. Por eso en su pipeline pueden tener tres escaladores, cada uno con su propia media, sin que se confundan entre sí.

Qué es self, con el trazado a la vista

La pregunta que responde: ¿sobre cuál objeto trabajo?

Si el método está escrito una sola vez en la clase, ¿cómo sabe sobre cuál objeto operar?

```
c3.sumar(7)           # lo que ustedes escriben
Contador.sumar(c3, 7) # lo que Python ejecuta por detrás

# Las dos líneas hacen exactamente lo mismo
```

La conclusión

self no tiene nada de mágico: es el primer parámetro de todo método, y es el espacio donde llega el objeto que está a la izquierda del punto.

Por eso se escribe en la definición y no se pasa al llamar: Python lo llena solo.

Dos tipos de atributo, y cuándo usar cada uno

Uno pertenece al molde; el otro, a cada copia

```
class Transformador:
    etiqueta = 'paso del pipeline'    # de CLASE: uno solo, compartido

    def __init__(self, columna):
        self.columna = columna        # de INSTANCIA: uno por objeto
```

Atributo de clase

Para constantes compartidas: una etiqueta, un valor por defecto, una lista de roles válidos. Cambiarlo afecta a todos los objetos.

Atributo de instancia

Para lo que cambia de un objeto a otro: la columna, la media aprendida, el estado. Es casi todo lo que usarán.

Regla práctica: si el valor puede ser distinto en dos objetos, va con self dentro de `__init__`.

¿Función o clase?

Una sola pregunta decide

¿El componente necesita recordar algo entre una llamada y otra?

No → una función basta

- Calcular la duración entre dos fechas
- Convertir texto a minúsculas
- Contar filas o valores únicos

Cada llamada es independiente y empieza de cero.

Sí → una clase se justifica

- Imputador: recuerda la mediana
- Codificador: recuerda el vocabulario
- Escalador: recuerda media y desviación

Aprenden en un momento y aplican después.

Prueba rápida: si el componente necesitaría una variable global para funcionar, esa variable debería ser un atributo.

Encapsulamiento: convertir un error silencioso en uno visible

Lo interno se protege y el uso incorrecto se detiene

```
class Escalador:
    def __init__(self, columna):
        self.columna = columna
        self._media = None      # interno
        self._ajustado = False  # interno

    def transformar(self, df):
        if not self._ajustado:
            raise RuntimeError(...)
```

El guion bajo

Señala lo interno. Python no lo impide: es un acuerdo entre quienes leen el código.

La guarda

Impide transformar antes de ajustar, que de otro modo daría un resultado sin sentido y sin aviso.

Lo que pasaría sin la guarda

El objeto no tiene media guardada, así que el cálculo daría error o, peor, un número. Números que parecen razonables y están mal. La guarda transforma eso en un mensaje que dice qué faltó.

Herencia: escribir una vez lo que se comparte

La clase base define el contrato; cada hija completa lo que cambia



Cada clase hija tiene ocho líneas propias y dispone de toda la maquinaria de control. Si mañana cambia el mensaje de error, se cambia en un solo lugar.

super(): cuando la hija necesita algo más

Llamar al constructor de la clase base sin repetirlo

```
class ImputadorFlexible(Transformador):  
  
    def __init__(self, columna, estrategia=None):  
        super().__init__(columna)          # ejecuta el __init__ de la base  
        self.estrategia = estrategia        # y agrega lo propio
```

Qué hace esa línea

Ejecuta el constructor de Transformador, que deja listos self.columna, self._parametros y self._ajustado.

Si la olvidan

El objeto queda sin esos atributos y falla más tarde, con un `AttributeError` que no apunta a la causa real.

Polimorfismo: tratar distinto por igual

Quien las usa no necesita saber de qué tipo es cada una

```
for paso in [ImputadorMediana('bmi'),  
             CodificadorNominal('work_type'),  
             EscaladorEstandar('age')]:  
    df = paso.ajustar(df).transformar(df)    # la misma llamada para los tres
```

Fíjense en lo que NO hay

Ningún if que pregunte de qué tipo es cada paso.

Funciona porque los tres ofrecen los mismos métodos.

La consecuencia práctica

Agregar un transformador nuevo no obliga a tocar el bucle.

En la rúbrica eso se llama bajo acoplamiento.

Los tres principios, juntos, en su pipeline

Así se declaran en el informe

1

Encapsulamiento

`_parametros` y `_ajustado` son internos, y `transformar()` falla si no se ajustó antes.

2

Herencia

Las clases hijas heredan de `Transformador` y no repiten el control de estado.

3

Polimorfismo

El pipeline recorre los pasos llamando la misma interfaz, sin preguntar el tipo.

El criterio de POO vale seis puntos y pide señalar dónde está cada uno en SU código. Estas tres frases, con el fragmento al lado, lo resuelven.

Cinco errores típicos al empezar con clases

Los va a cometer, y conviene reconocerlos rápido

-  **Olvidar self en la definición del método**
Python reclama un argumento de más. El objeto llegó, pero no había dónde recibirlo.
-  **Escribir `_init_` con un solo guion bajo**
No se ejecuta y nadie avisa. El objeto queda sin atributos.
-  **Modificar el DataFrame que recibieron**
Usen `df.copy()`. Si no, el objeto del usuario cambia sin que él lo pidiera.
-  **Crear clases que no recuerdan nada**
Si no hay estado entre llamadas, era una función. La clase solo agrega ruido.
-  **Olvidar `super().__init__()` en la hija**
Los atributos de la base nunca se crean, y el error aparece mucho más tarde.

Cohesión y acoplamiento

Las dos palabras del descriptor que casi nadie explica

Alta cohesión

Cada componente hace UNA cosa. Si al describir un archivo hace falta la palabra «y» varias veces, la cohesión es baja.

Bajo acoplamiento

Los componentes dependen poco unos de otros. Cambiar uno no obliga a tocar los demás.

La prueba que resuelve el asunto sin teoría

Si cambiamos la forma de imputar, ¿cuántos archivos hay que modificar? Si es uno, el acoplamiento es bajo. Si son tres, el diseño es frágil.

```
src/carga.py · src/transformadores.py · src/pipeline.py · src/medicion.py  
# cada archivo con un propósito que se enuncia en una frase
```

04

Patrones de diseño

Nombres para soluciones que ustedes ya usaron en la Fase 2

Strategy: varias formas de hacer lo mismo

Separar el QUÉ se hace del CÓMO se hace

El problema

Compararon imputar con mediana, media o por grupo. Si cada forma es un bloque distinto, cambiar obliga a reescribir.

La solución

La clase no sabe imputar: sabe cuándo. El cómo lo aporta la estrategia que recibe como parámetro.

```
for estrategia in [PorMediana(), PorMedia(), PorMedianaDeGrupo('gender')]:  
    salida = ImputadorFlexible('bmi', estrategia).ajustar_transformar(datos)  
    # ... medir el efecto sobre la dispersión
```

Con Strategy, la tabla comparativa que pide el criterio de preprocesamiento cuesta un bucle.

Factory y Observer

Decidir qué construir, y registrar sin estorbar

Factory

Concentra en un solo lugar la decisión de qué transformador corresponde a cada rol.

```
crear_transformador('continua', 'bmi')  
crear_transformador('nominal', 'region')
```

```
# el diccionario de F1 gobierna  
# el pipeline
```

Observer

El pipeline avisa cada paso; quien quiera registrar se suscribe. No sabe quién escucha.

```
pipe.suscribir(Bitacora())  
pipe.suscribir(ReporteConsola())
```

```
# la bitácora es producto de la  
# ejecución, no un texto aparte
```

Con Observer, la bitácora de decisiones deja de escribirse a mano después de ejecutar. Se genera cada vez que el pipeline corre, así que no puede quedar desactualizada.

Singleton, y cuál usar

Un patrón bien justificado vale más que cuatro mencionados

Singleton

Una sola configuración compartida: semilla, rutas y umbrales. Evita que dos partes del pipeline usen semillas distintas.

Con su advertencia

Introduce estado global, que complica las pruebas. Úsenlo solo para configuración.

Strategy

Varias formas de hacer lo mismo

Factory

Elegir qué construir según una condición

Observer

Registrar sin que el ejecutor sepa quién anota

Singleton

Una sola configuración compartida

No usen los cuatro. Usen el que su proyecto necesita y expliquen por qué.

El resultado: cómo queda el proyecto

El mismo trabajo, con otra arquitectura

```
# Antes (Fase 2)
df = pd.read_csv('datos.csv')
df = df.drop(columns=['id'])
m = df['bmi'].median()
df['bmi'] = df['bmi'].fillna(m)
df = pd.get_dummies(df, ['work_type'])
media = df['age'].mean() ...
```

```
# Después (Fase 3)
pipeline = construir_pipeline()
pipeline.ajustar(entrenamiento)
ent = pipeline.transformar(entrenamiento)
pru = pipeline.transformar(prueba)

# mismo resultado, otra arquitectura
```

Y la comprobación que más rinde

```
pd.testing.assert_frame_equal(version_f2, version_f3)
```

Si el pipeline con clases produce exactamente el mismo conjunto que el código de la Fase 2, tienen una prueba objetiva de que la reorganización no rompió nada. Y si no coincide, encontraron un error antes de entregarlo.

Qué entrega la Fase 3 a la Fase 4

Cuatro artefactos, y una advertencia

entrenamiento_f3.csv

El conjunto procesado

prueba_f3.csv

Con el mismo esquema

diccionario_f3.csv

Columna, tipo y origen

parametros_f3.csv

Lo que aprendió cada paso

Lo que la Fase 4 NO debe hacer

Volver a imputar, codificar o escalar. Sobre un conjunto ya transformado, escalaría dos veces.

Lo que sí debe hacer

Leer, explorar, visualizar y comunicar. Y si necesita una transformación nueva, agregarla al pipeline de la Fase 3.

Esa es la ventaja concreta de haber modularizado: la Fase 4 hereda una herramienta, no un archivo suelto.

Lo que más cuesta puntos

Observado en las entregas de la Sumativa 1



Notebooks sin ejecutar completos

Numeración discontinua o salidas borradas. Se resuelve con Restart & Run All antes del último commit.



El informe afirma lo que el repositorio no muestra

Si declaran que algo se ejecutó, la evidencia tiene que estar en el artefacto versionado.



src/ declarada y vacía

El código sigue en celdas. No se puede refactorizar a clases lo que no está en módulos.



Identidades de Git duplicadas

Una persona aparece como dos autores. Los criterios de repositorio exigen contribuciones trazables.

Para esta semana

Cuatro pasos y tienen el esqueleto de la entrega

- 1 Conviertan UN paso de su pipeline en una clase que herede de Transformador
- 2 Midan UNA operación de dos formas, con la comprobación de que dan lo mismo
- 3 Decidan si su proyecto justifica recursividad, y anoten por qué
- 4 Ejecuten `assert_frame_equal` contra su conjunto de la Fase 2

Material de la semana: [cuaderno f3_s02_guia_poo.ipynb](#) · guías de apoyo de la Formativa 3 y la Sumativa 2 · foro técnico

omar.salinas@unab.cl